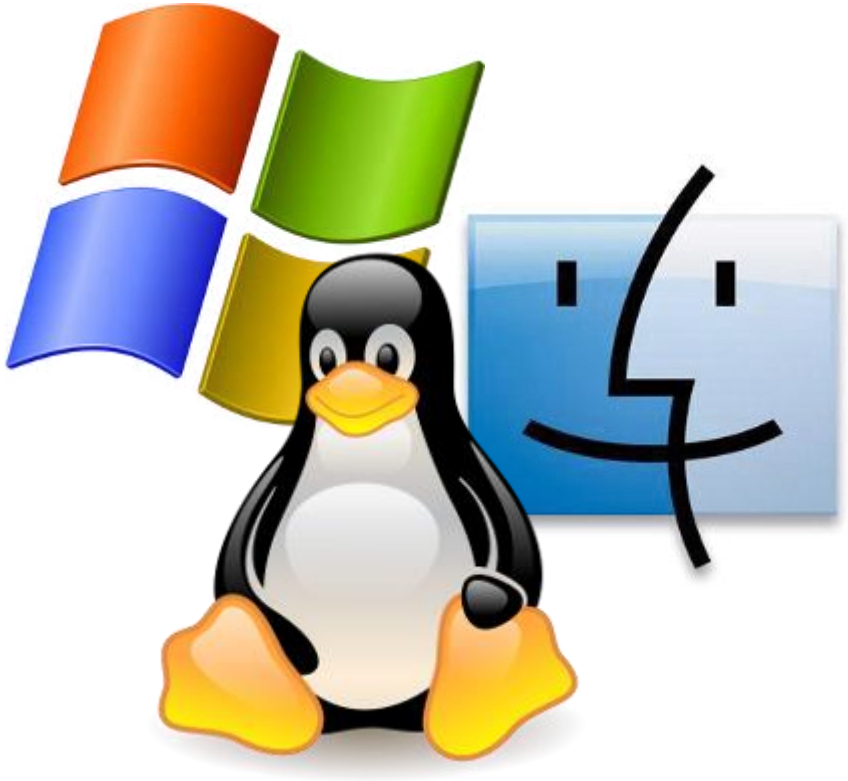
5 – Ein- und Ausgabeverwaltung



Tobias Werner M. Sc.
Uni Bayreuth – Angewandte Informatik 3
Leibniz-FH Hannover

tobias.werner.doz@leibniz-fh.de

(in Teilen nach der Vorlesung
"Betriebssysteme" von Prof. Dominik Henrich
an der Universität Bayreuth)

[Logos von Windows 7, Linux und Mac OSX]

Vortragsübersicht

Heranführung

- Was für Geräte sind an Ihrem Computer angeschlossen?
- Was ist USB?
- Was ist eine Schnittstelle?

- Sie haben in der Firma eine Aufgabe zu erledigen.
  Ein anderer Mitarbeiter fragt Sie ständig, ob Sie schon fertig sind,
  weil er auf ihr Ergebnis wartet.
  → Wo liegt der Nachteil an diesem Vorgehen?
  → Hat das Vorgehen auch einen Vorteil?
  → Wie könnte man alternativ (und womöglich besser) vorgehen?

Kompetenzen

- an der Ein- und Ausgabe beteiligte Hardware-Komponenten kennen
- Software-Konzepte zur Ein- und Ausgabe verstehen
- die OS-Struktur für Ein- und Ausgabe verstehen

Vortragsübersicht

5 – Ein- und Ausgabeverwaltung

- E/A-Hardware
- E/A-Software
- Architektur für E/A-Software
- Konkrete Peripheriegeräte

E/A-Hardware (1)

Aufgaben des Betriebssystems

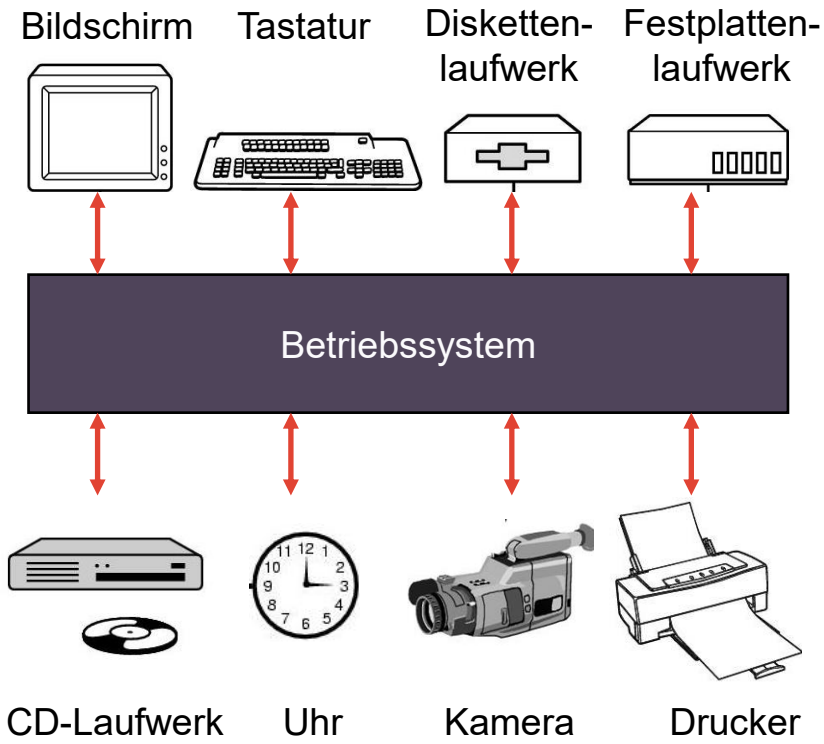
- Schnittstelle zwischen Geräten und Rest zur Verfügung stellen
- Steuerung und Überwachung aller E/A-Geräte, insbesondere:
  - → Kommandos an Geräte weiterleiten
  - → Unterbrechungen abfangen
  - → Etwaige Fehler behandeln

Aufgabenstellung:

- Wie gelangen Daten von einem Peripheriegerät in den Hauptspeicher oder umgekehrt?

Abkürzungen:

- E/A: Ein- und Ausgabe
- IO: Input and Output



aus [Tanenbaum15]

E/A-Hardware (2)

E/A-Geräte

übliche Klassifikation der E/A-Geräte:

- *blockorientierte Geräte* (engl. *block devices*):
  Blöcke einzeln, unabhängig adressierbar
- *zeichenorientierte Geräte* (engl. *character devices*):
  reine Zeichenströme

→ Klassifikation ist unvollständig
(z. B. fehlen analoge Ausgaben)
→ Klassifikation dennoch anwendbar
für geräteunabhängigen SW-Teil im OS
→ geräteabhängiger SW-Teil (Gerätetreiber)
realisiert die Umsetzung

- große Unterschiede der Geräte bei den Datenraten
  (siehe Übersicht, 2009)

Ein-/Ausgabegerät	Übertragungsrate
Tastatur	80 Bit/s
Maus	800 Bit/s
56K Modem	56 KBit/s
Scanner	3,2 MBit/s
Digitaler Camcorder	28 MBit/s
WLAN nach 802.11g	54 MBit/s
52-fach CD-ROM	62 MBit/s
Fast Ethernet	54 MBit/s
Compact Flash Card	320 MBit/s
FireWire (IEEE 1394)	400 MBit/s
USB 2.0	480 MBit/s
SONET OC-12 Netzwerk	624 MBit/s
Ultra-2-SCSI	640 MBit/s
Gigabit Ethernet	1 GBit/s
SATA-Plattenlaufwerk	2,4 GBit/s
Ultrium Bandlaufwerk	2,5 GBit/s
PCI Bus	4,2 GBit/s
USB 3.0	4,8 GBit/s

aus [Tanenbaum15]

E/A-Hardware (3)

Hardware-Ansteuerung

***Gerät* (engl. *device*):**

extern vorhandene, früher oft mechanische Komponente eines Rechners

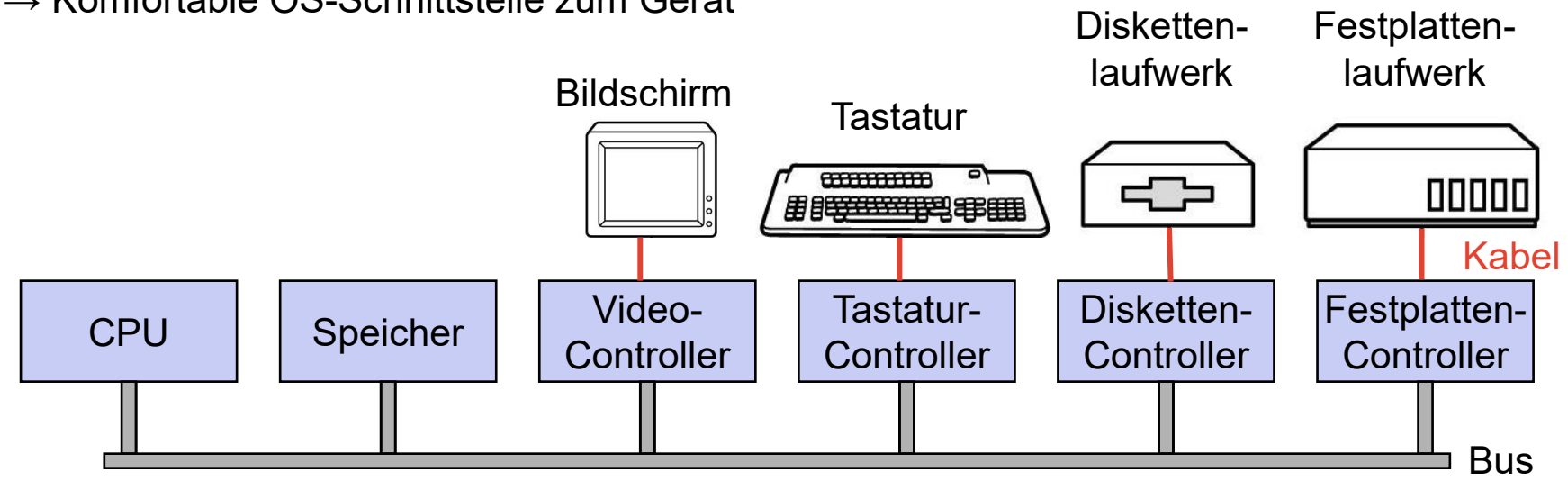
***Steuereinheit* (engl. *controller*):**

elektrische Komponente, die ein oder mehrere identische Geräte steuert

→ Schnittstelle zwischen Controller und Gerät meist standardisiert

(z. B. PCI, SCSI, SATA, USB, IDE, ...)

→ Komfortable OS-Schnittstelle zum Gerät



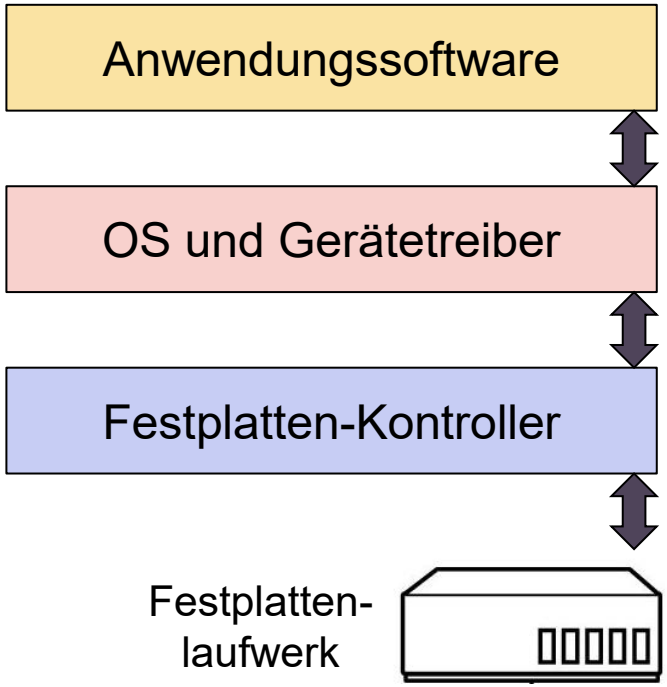
aus [Tanenbaum15]

E/A-Hardware (4)

Beispiel Festplatten-Kontroller

Aufgaben:

- konvertiert zwischen seriellem Bitstrom und Blöcken aus Bytes
- führt bei Bedarf Fehlerkorrektur durch
- stellt Ergebnis im Hauptspeicher zur Verfügung



"Öffne Datei `test.txt` und lies 15 Bytes ein, speichere das Ergebnis an Adresse `0x003AE21F`."

"Gehe zu Byte 50321 der Festplatte und lies von dort 15 Bytes ein, schicke die Bytes über den Bus zurück."

"Lege eine Spannung von 15 mV an den Elektromotor für die Drehspindel und keine Spannung an den Schreibkopf. Warte 10 ms, dann prüfe die Spannung am Lesekopf, ..."

E/A-Hardware (5)

Kommunikation mit E/A-Geräten

aus [Tanenbaum15]

- OS steuert Gerät durch Schreiben und Lesen von Controller-Register
  bzw. von zusätzlichem Datenpuffer

***port-basierte E/A*:**
Kommunikation mit Register über spezielle Port-Befehle, z. B.

```
IN Reg3, Port7
```

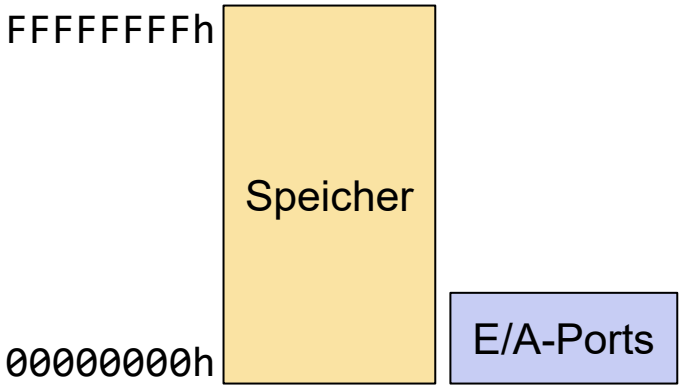
- nur über direkte Maschinenbefehle
- getrennte Adressräume

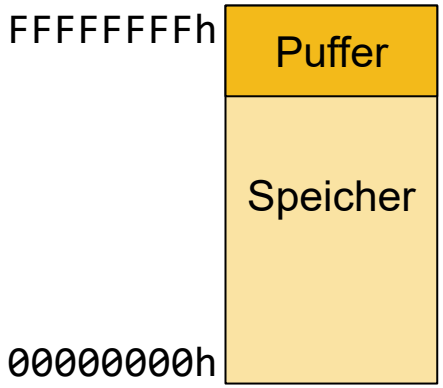
***speicher-basierte E/A*:**
Register sind in zuvor nicht verwendeten Hauptspeicher-bereich eingeblendet

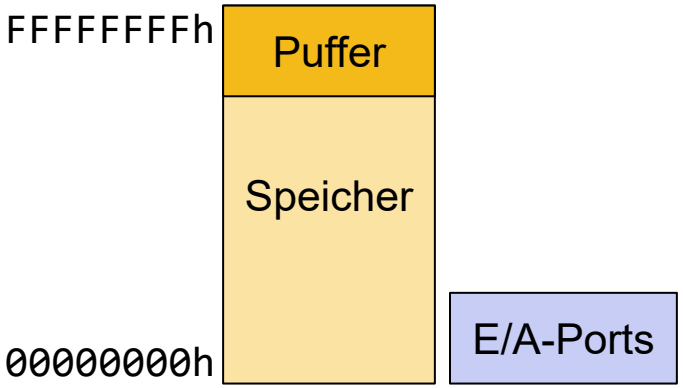
- reine Hochsprache möglich
- Schutz über virt. Speicher
- gemeinsamer Adressraum

***hybride E/A*:**
E/A-Ports für Controller-Register und speicher-abgebildete E/A über Datenpuffer

- getrennte Adressräume



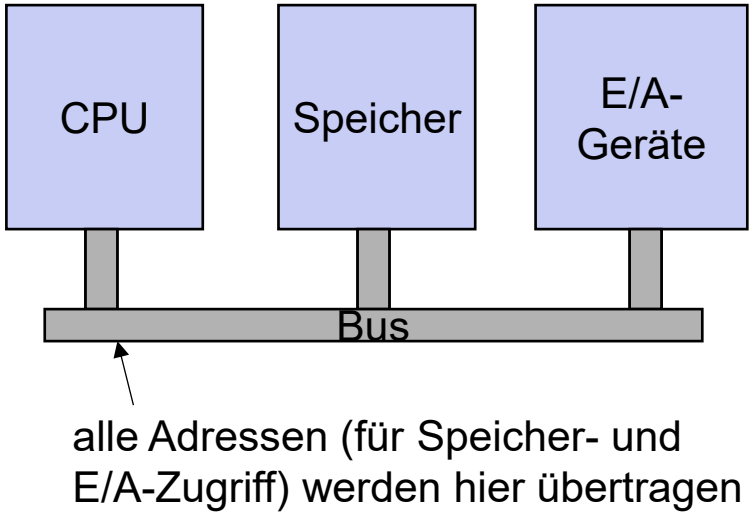




E/A-Hardware (6)

Probleme der speicherbasierten E/A

- das Caching der Daten muss für abgebildeten Speicherbereich (selektiv) ausgeschaltet sein
- bei Ein-Bus-Architekturen müssen alle Geräte und Speichermodule immer den Adressbus abhören
- bei Zwei-Bus-Architekturen können die Geräte nicht auf die eingeblendeten E/A-Adressen reagieren



aus [Tanenbaum15]

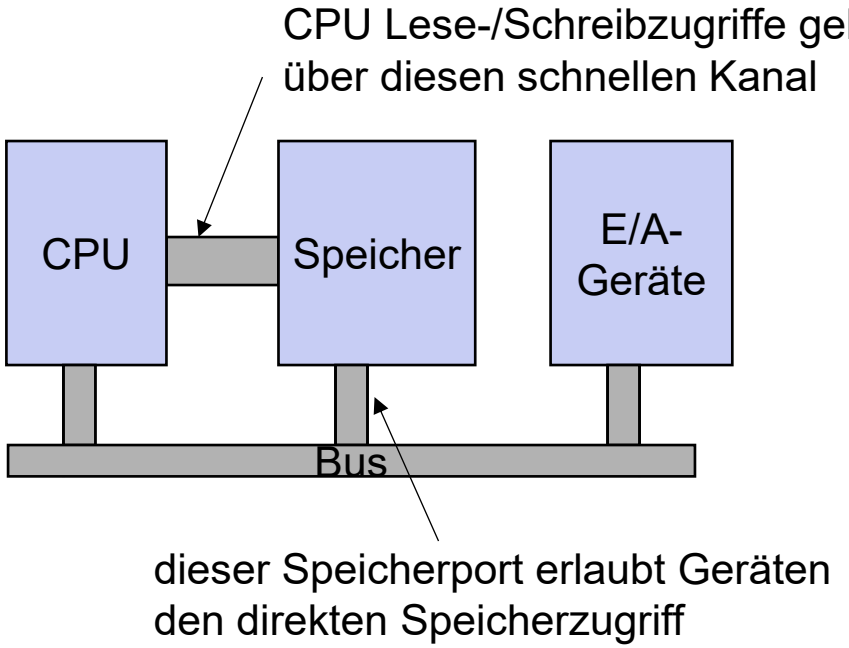
E/A-Hardware (7)

Lösungen bei Zwei-Bus-Architekturen

- CPU überträgt E/A-Befehl auf E/A-Bus, falls Speicher nach Time-Out nicht reagiert,

oder

- zusätzliche Hardware (z. B. die PCI-Brücke) überträgt die E/A-Adressen
  vom Speicher-Bus auf den E/A-Bus (z. B. bei Pentium-CPU)



aus [Tanenbaum15]

E/A-Hardware (8)

aus [Tanenbaum15]

Direkter Speicherzugriff (engl. *direct memory access, DMA*)

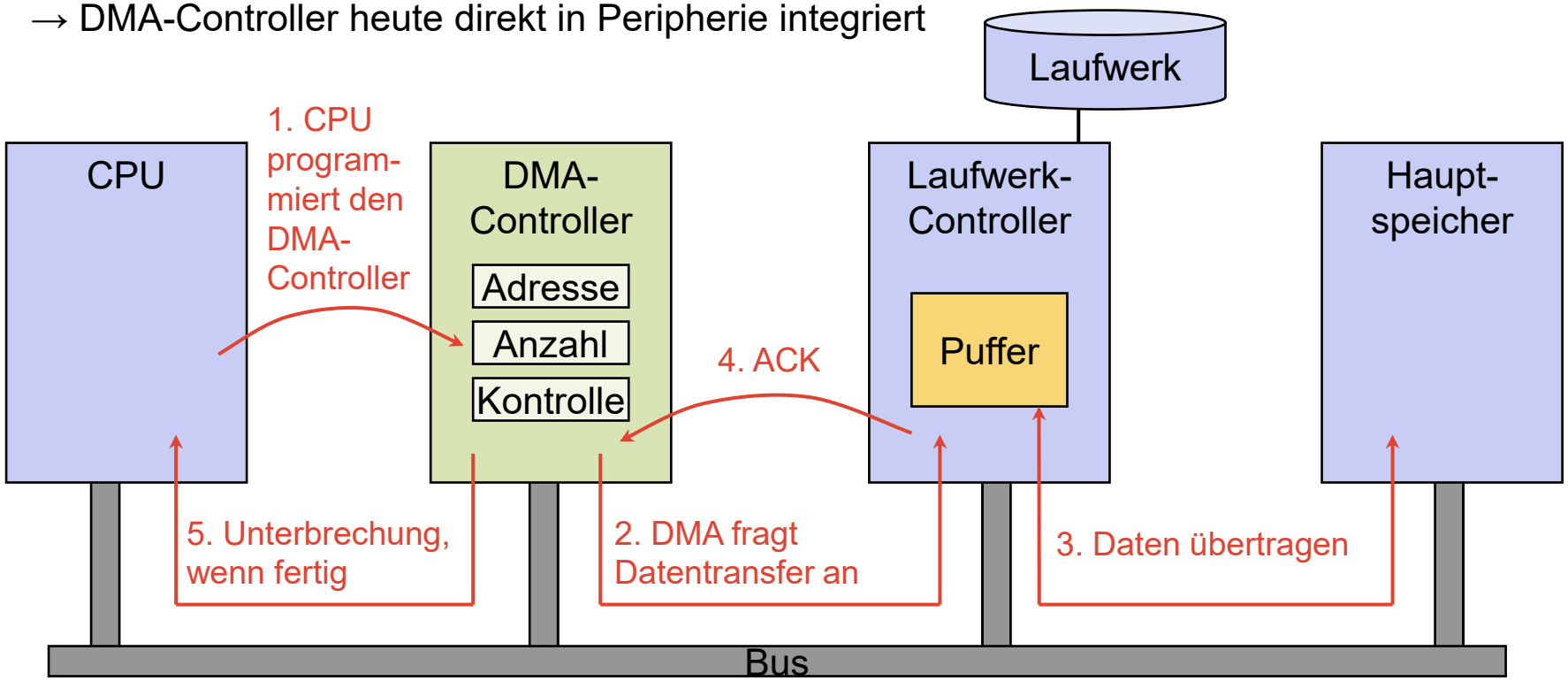
Problem: **w**ortweise Datenübertragung vom/zum Gerät durch CPU kostet viel CPU-Zeit

Lösung: DMA-basierte Übertragung mit separatem DMA-Controller

→ bei DMA-Übertragung wird DMA und Laufwerk-Controller durch CPU initialisiert (Schritt 1)

→ dann stößt DMA zyklisch Übertragung der Worte an (Schritte 2 bis 4)

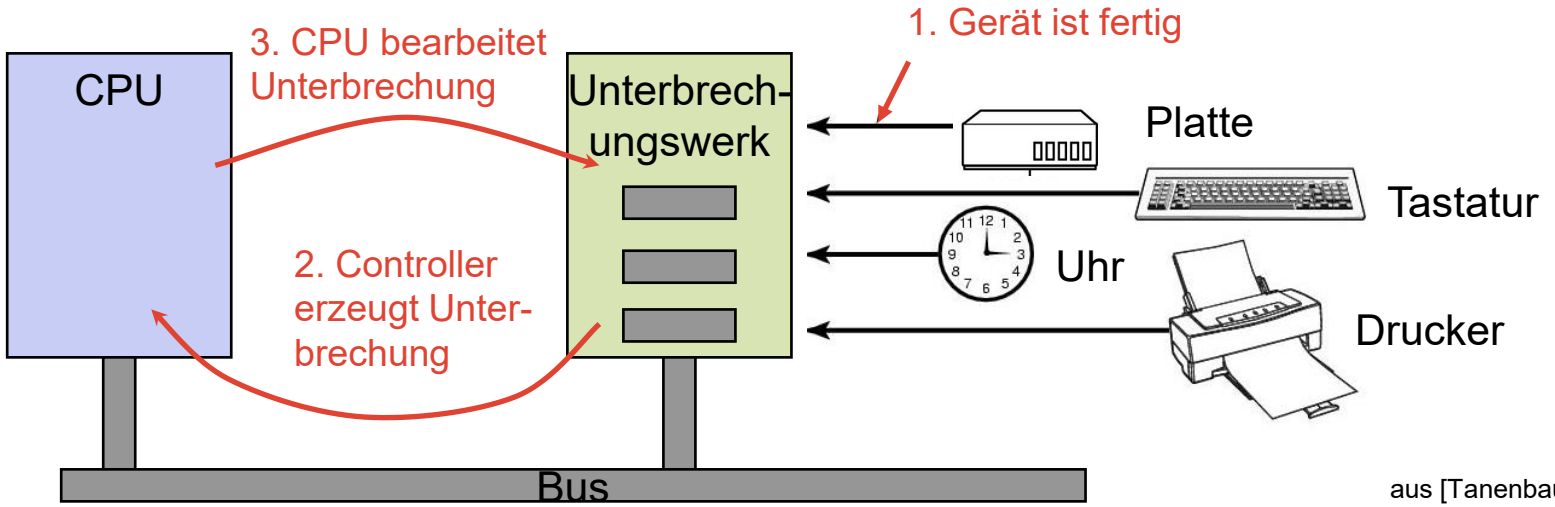
→ DMA-Controller heute direkt in Peripherie integriert



E/A-Hardware (9)

Unterbrechungswerk

1. (Geräte- bzw. DMA-) Controller signalisieren der CPU ein Ereignis
   (z.B. E/A-Ende) mittels einer Unterbrechung
   → Unterbrechungswerk (Interrupt-Controller) wählt Unterbrechung
   mit höchster Priorität aus und speichert die anderen
2. Unterbrechungswerk legt Geräte-ID (*Unterbrechungsvektor*)
   auf E/A-Bus und unterbricht CPU
3. CPU behandelt Unterbrechung durch eigene Routine
   (analog zu Systemaufrufen, Traps)



aus [Tanenbaum15]

E/A-Hardware (10)

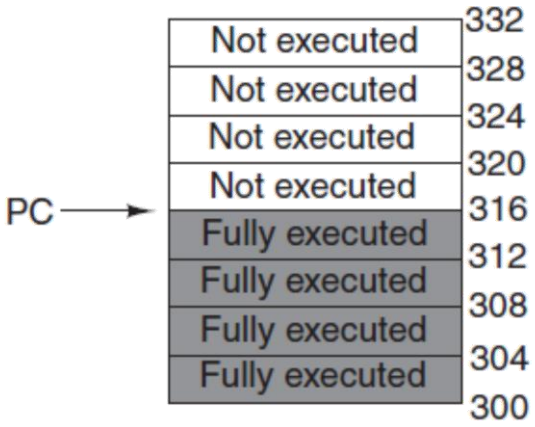
Varianten der Unterbrechung

***präzise Unterbrechung*:**

Unterbrechung mit 4 Eigenschaften:

- *Befehlszähler* (PC) ist gespeichert
- alle Befehle vor dem PC sind beendet
- kein Befehl nach dem PC ist beendet
- Stand der Ausführung des PC-Befehls ist bekannt

→ Befehle nach PC dürfen zwar angefangen sein, aber alle Änderungen von Register / Speicher müssen rückgängig gemacht sein

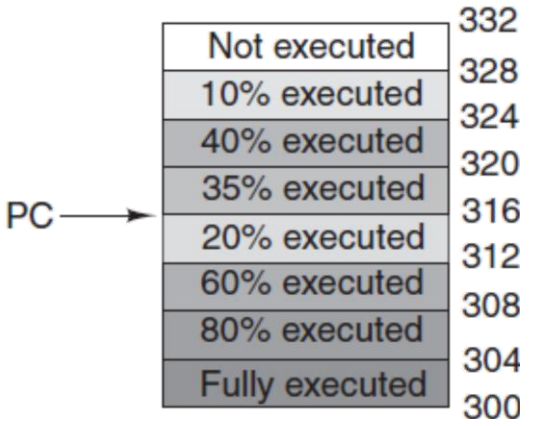


unpräzise Unterbrechung:

Unterbrechung mit den Befehlen um den PC haben unterschiedlichen Ausführungsstand

→ CPU speichert Zustands auf dem Stapel

→ verlangsamt die Behandlung der Unterbrechung



aus [Tanenbaum15]

Vortragsübersicht

5 – Ein- und Ausgabeverwaltung

- E/A-Hardware
- E/A-Software
- Architektur für E/A-Software
- Konkrete Peripheriegeräte

E/A-Software (1)

Ziele der E/A-Software

geräteunabhängigkeit:

- jedes Programm kann auf Daten von allen E/A-Geräten gleichartig zugreifen
- E/A-Gerät muss dazu nicht vorher spezifiziert werden (z. B. Datenträger, CD, Bildschirm, ...)

Benennungsschemata:

- einheitliche Datei- bzw. Geräte-Namen bestehend aus Zeichenkette oder Nummer
- unabhängig von der (konkreten) Hardware

Fehlerbehandlung:

- Fehler sollten möglichst nahe an der Hardware behandelt werden (Verursacherprinzip)

E/A-Software (2)

synchrone Programmierung:

- Übertragung sollte synchron programmiert und asynchron ausgeführt werden

Datenpufferung:

- ermöglicht Analyse der übertragenen Daten
- ermöglicht Entkopplung unterschiedlicher Übertragungsraten
- bringt aber Kopieren der Daten mit sich

flexible Gerätenutzung:

- OS sollte Prozessen gemeinsame und exklusive Nutzung von Geräten ermöglichen
- ermöglicht Deadlocks

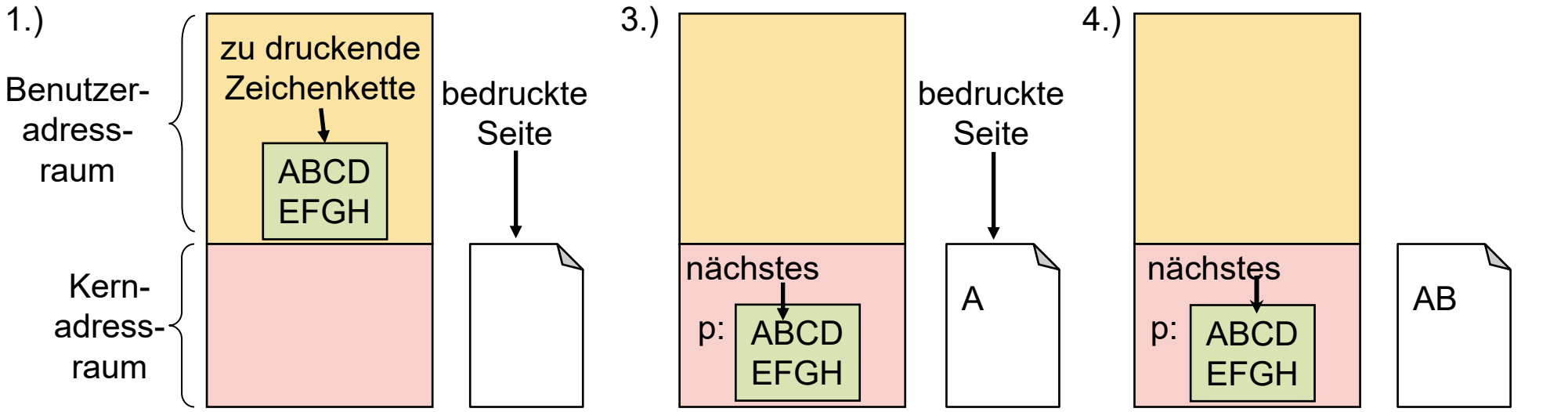
E/A-Software (3)

Beispielaufgabe für E/A

Anwendung möchte Zeichenkette `ABCDEFGH` der Länge `count` im Puffer `buffer` ausdrucken

Schritte zur Vorbereitung:

1. Benutzerprozess versucht, Gerät zu belegen und wird ggf. blockiert
2. Prozess übergibt per Systemaufruf Daten an Druckertreiber
3. OS kopiert (zumeist) die Daten in eigenen Adressraum in Puffer `p[ ]`
4. ... (hier 3 Alternativen auf den nächsten Folien)



aus [Tanenbaum15]

E/A-Software (4)

Programmgesteuerte E/A (polling)

solange bis alle Daten übertragen:

- OS prüft Gerät, bis es bereit ist
- OS sendet nächstes Datum

  → einfache Programmierung

  → OS ist blockiert durch busy waiting

Systemaufruf:

```
copy_from_user(buffer, p, count);                // p ist der Puffer im OS-Kern
for (int i = 0; i < count; i++) {                // Schleife über alle Zeichen
     while (*printer_status_reg != READY) { }    // warte bis Drucker bereit
     *printer_data_register := p[i];             // gebe ein Zeichen aus
}
return_to_user();
```

E/A-Software (5)

Unterbrechungsgesteuerte E/A

- Systemaufruf schickt 1. Datum an Gerät, sobald es frei ist
- OS blockiert Anwendungsprozess bis zum Ende der Übertragung
- sobald das Gerät fertig ist, unterbricht es die CPU
- OS behandelt Unterbrechung und sendet nächstes Datum

  → nach Übertragungsende wird Anwendungsprozess wieder bereit gestellt

  → keine Blockierung des OS

  → hohe CPU-Belastung, da für jedes Datum eine Unterbrechung notwendig

Systemaufruf:

```
copy_form_user(buffer, p, count);
enable_interrupts();
while (*printer_status_reg !=READY)
    { }
i := 0;
*printer_data_register := p[i];
count := count - 1;
scheduler();
```

Unterbrechungsbehandlung:

```
if (count > 0) {
     i = i + 1;
     *printer_data_register := p[i];
     count := count - 1;
}
else unblock_user();

acknowledge_interrupt();
return_form_interrupt();
```

E/A-Software (6)

DMA-gesteuerte E/A

- DMA-Controller schreibt Daten direkt in Puffer des Geräte-Controllers
- DMA führt „programmgesteuerte E/A“ durch
- gesamte Arbeit von CPU auf DMA verschoben

  → nur eine Unterbrechung pro Pufferfüllung

  → keine Verschwendung von CPU-Zeit

  → DMA-Controller langsamer als CPU

  → falls CPU unterbeschäftigt, dann ist programm- bzw. unterbrechungs-gesteuerte E/A schneller

Systemaufruf:

```
copy_form_user(buffer, p, count);
set_up_DMA_controller(...);
scheduler();
```

Unterbrechungsbehandlung:

```
acknowledge_interrupt();
unblock_user();
return_form_interrupt();
```

Vortragsübersicht

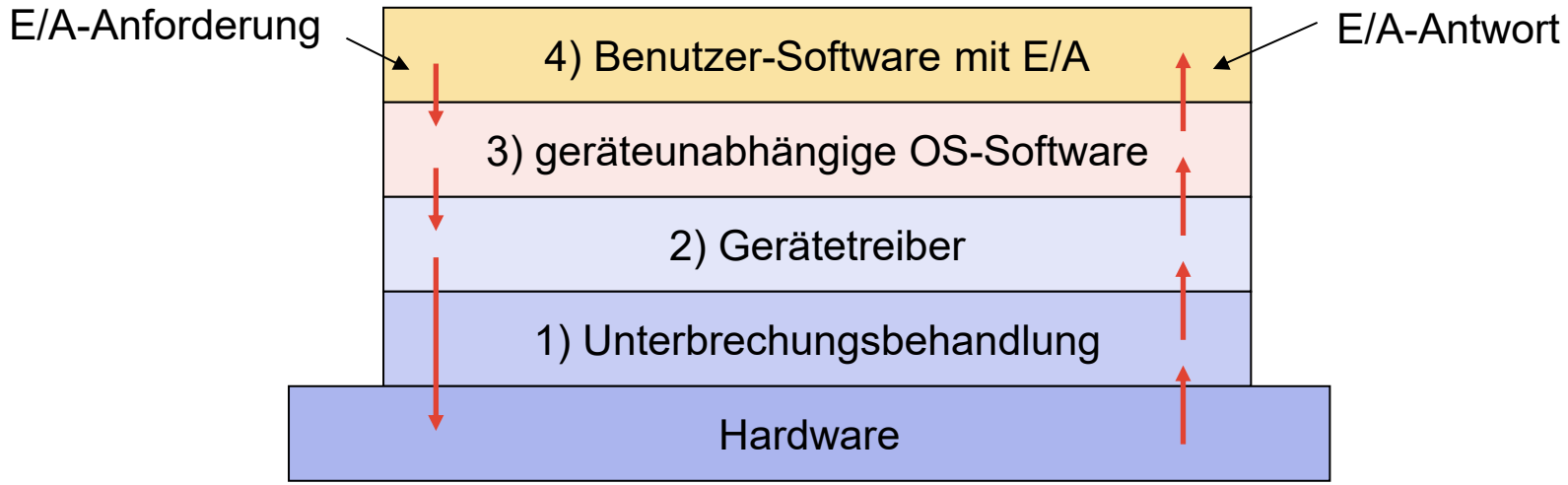
5 – Ein- und Ausgabeverwaltung

- E/A-Hardware
- E/A-Software
- Architektur für E/A-Software
- Konkrete Peripheriegeräte

Architektur für E/A-Software (1)

Schichtenmodell der E/A

- eine Funktionalität pro Schicht
- wohl-definierte Schnittstellen zwischen den Schichten
- hier: maschinen-unabhängige Beschreibung



Architektur für E/A-Software (2)

1) Unterbrechungsbehandlung

- Unterbrechungsbehandlung wird am besten vor Anwendung versteckt
- dazu kann E/A-Operation Anwendung blockieren, bis Unterbrechung E/A-Ende anzeigt
- Unterbrechungsbehandlungs-Routine führt E/A-Operation durch und deblockiert die Anwendung, welche die E/A-Operation angestoßen hat

- weitere notwendige OS-Schritte zur Behandlung einer Unterbrechung (Reihenfolge maschinenabhängig):

Unterbrechung für E/A sei eingetroffen:

1. sichere Register, welche nicht schon durch HW gesichert wurden
2. Erzeuge Kontext für Unterbrechungs-Behandlungs-Routine (TLB, MMU, Paging, ...)
3. erzeuge Stapel für diese Routine
4. andere Unterbrechungen wieder zulassen
5. Registerinhalte von Sicherung (z.B. Stapel) in Prozesstabelle kopieren
6. führe eigentliche E/A-Behandlungsroutine durch
7. wähle nächsten Prozess
8. initialisiere MMU-Kontext für nächsten Prozess (inkl. TLB, Cache)
9. lade die neuen Prozessregister
10. starte den neuen Prozess

Architektur für E/A-Software (3)

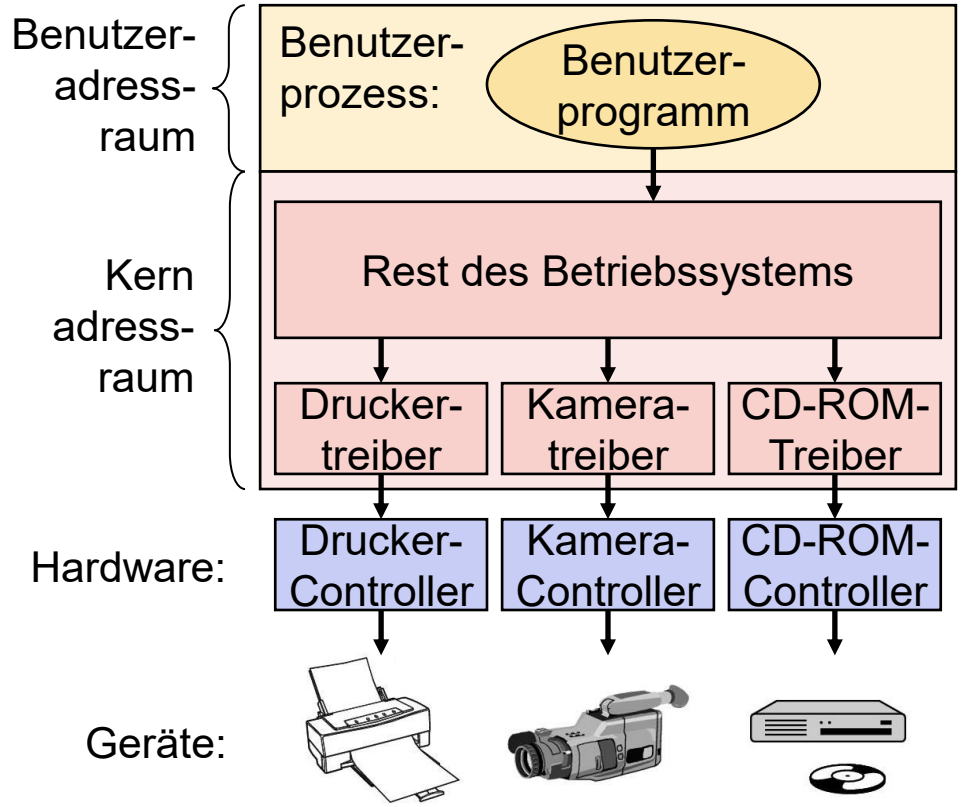
aus [Tanenbaum15]

2) Gerätetreiber

- Gerätetreiber ist die geräteabhängige Steuerungs-Software des OS für Gerät
- Geräte bzw. Controller unterscheiden sich in ihrer Funktionalität sehr
  → für jeden Gerätetyp und jedes OS eigene Treiber vom Geräte-Hersteller
- teilweise laufen Treiber im OS-Kern
  → Treiber-Installation im OS nötig
  → OS-Absturz durch Treiber möglich

sinnvolle Treiber-Funktionen:

- Gerät initialisieren
- Befehle ausführen
- Gerät kontrollieren
- Fehler melden
- wiedereintrittsfähig (*reentrant*)
- Geräte zur Laufzeit an/absteckbar (*hot pluggable*)



Architektur für E/A-Software (4)

Beispiel USB-Schnittstelle

Hardware:

- USB 3.0: 2 Strom- und 7 bit-serielle Datenleitungen, 4,8 GBit/s (duplex)
- USB 3.1: 10 Gbit/s Datenrate, 100 Watt Power Delivery, symm. Typ-C-Stecker
- zentraler Host-Controller koordiniert ≤127 sternförmig verbundene Geräte
- durch Verteiler (Hubs) entsteht physikalisch ein Baum mit logischen Punkt-zu-Punkt-Verbindungen
- hot pluggable

Software:

- generische Treiber für Geräteklassen
- individuelle Treiber sind nachladbar
- Übertragungsmodi:
  - isochroner Transfer (feste Datenrate)
  - Interrupt-Transfer (begrenzte Latenz)
  - Bulk-Transfer (restliche Bandbreite)
  - Control-Transfer (kurze Nachrichten)







Klasse	Verwendung	Beschreibung	Beispiele
00h	Gerät	unspezifiziert	die Klasse wird auf Ebene der Interface-Deskriptoren definiert
01h	Interface	Audio	Lautsprecher, Mikrofon, Soundkarte, MIDI
02h	Beides	Kommunikation	Netzwerkkarte, Modem, Adapter für serielle Schnittstelle
03h	Interface	HID	Tastatur, Maus, Joystick, etc.
...	...	...	...

Architektur für E/A-Software (5)

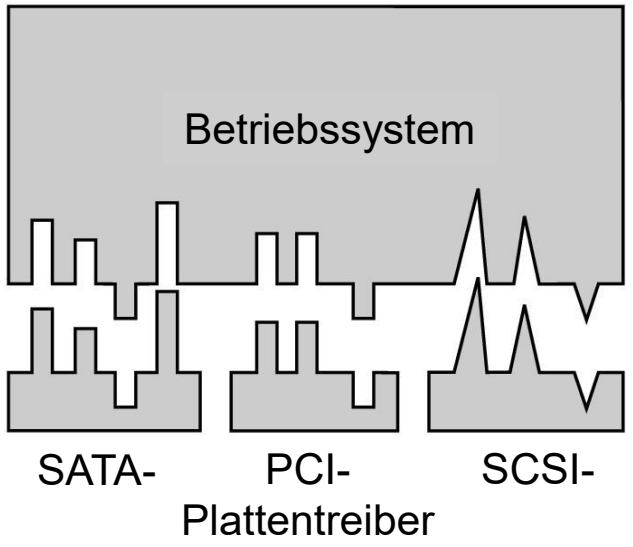
3) Geräteunabhängige E/A-Software

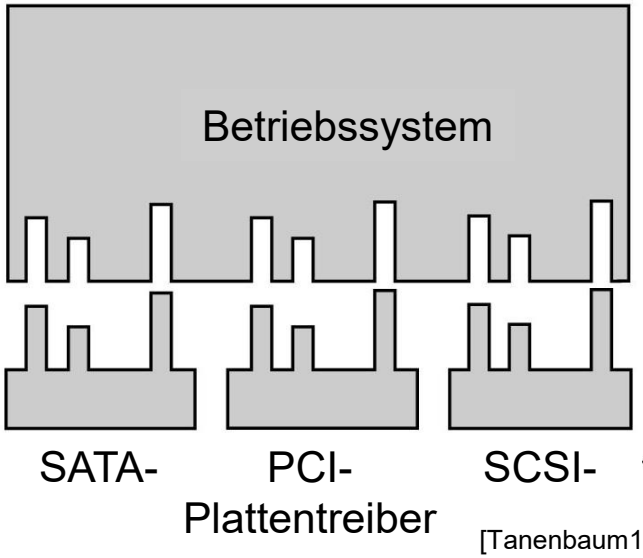
Grenze zwischen geräteabhängiger
und geräteunabhängiger Software
ist vom Gerät und vom OS abhängig

Funktionalität der geräteunabhängigen OS-Software:

- uniforme Schnittstelle für Treiber
  (z. B. als Datei im Dateisystem)
  → reduziert OS-Anpassungsbedarf
  → erleichtert OS- und Treiber-Programmierung
  → Geräte-Funktionalität nicht voll genutzt

- Pufferung
- Fehlerbericht
- Anforderung und Freigabe von Geräten
- geräteunabhängige Blockgrößen
  → die nächsten Folien



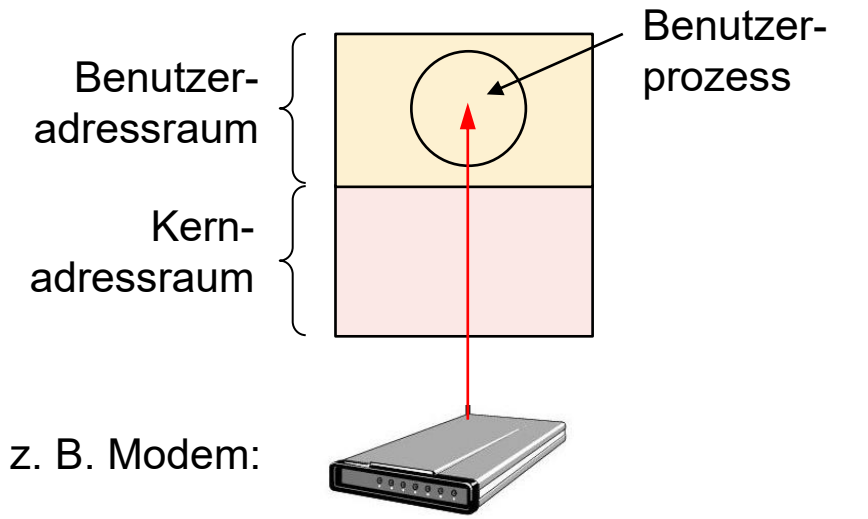


[Tanenbaum15]

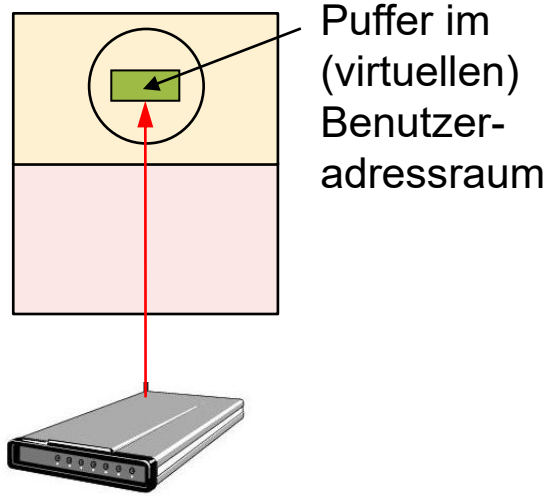
Architektur für E/A-Software (6)

Alternativen der Pufferung

ungepufferte E/A:
Anwendung wird blockiert
bis zur E/A des nächsten Zeichens
→ ineffizient



Pufferung im Benutzerraum:
E/A von n Zeichen in/aus Benutzerpuffer
→ Puffer ist nicht auslagerbar



aus [Tanenbaum15]

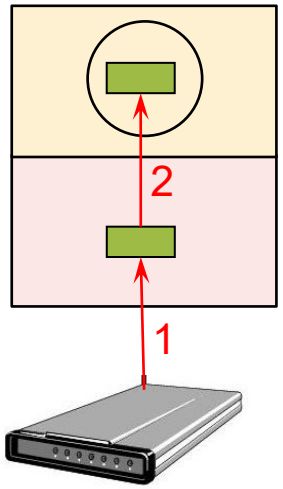
Architektur für E/A-Software (7)

Pufferung im OS-Kern:

E/A von n Zeichen in/aus OS-Kern-Puffer

und danach Kopie in/aus Benutzerraum

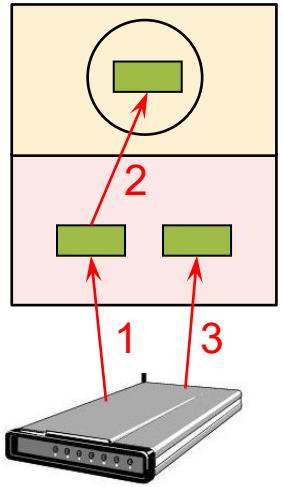
→ Kopieren u. E/A nicht gleichzeitig



Doppelte Pufferung:

E/A alternierend in/aus 2 OS-Puffern,

wobei der andere in/aus Benutzerraum kopiert wird



aus [Tanenbaum15]

Architektur für E/A-Software (8)

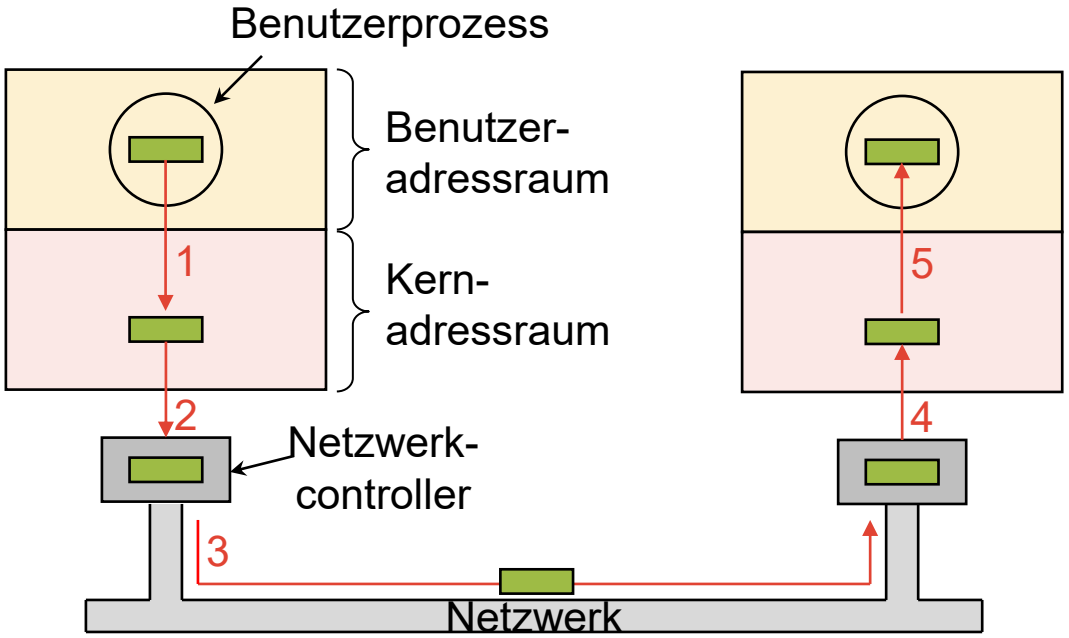
Nachteil der Pufferung

Pufferung bedeutet häufiges Kopieren der Daten

→ E/A-Operation wird langsam

Beispiel:

1. Kern kopiert Daten in Kern-Puffer
2. Treiber kopiert Daten in Controller
3. Controller schreibt Daten auf Bus
   und Empfänger puffert Daten
4. Treiber puffert Daten
5. Anwendung puffert Daten



aus [Tanenbaum15]

Architektur für E/A-Software (9)

Weitere Funktionen geräte-unabhängiger OS-Software

Fehlerbericht:

- Rahmenwerk der Fehlerbehandlung ist geräteunabhängig
  → bei Software-Fehler ist eine Fehler-Nummer an Anwendungsprozess weiterzugeben
  → bei Hardware-Fehler muss der Treiber die Reaktion bestimmen

Anforderung und Freigabe von Geräten:

- Zugriff auf exklusiv benutzbare Geräte muss das OS absichern
  (z. B. über das Dateisystem)

geräteunabhängige Blockgrößen:

- unterschiedliche Blockgrößen der Geräte werden versteckt
- mehrere Blöcke können zu einem Block zusammengefasst werden

Architektur für E/A-Software (10)

4) Benutzer-Ein-/Ausgabe-Software

- Bibliotheks-Routinen zur Unterstützung bei der Programmierung

  Beispiel in C:

  ```
  count = write(fd, buffer, nbytes);
  ```

- Bibliotheks-Routinen zur Formatierung der E/A

  Beispiel in C:

  ```
  printf("Das Quadrat von %3 ist %6d.\n", i, i * i);
  ```

  ergibt für `i = 2` die Ausgabe:

  ```
  Das Quadrat von 2 ist 4.
  ```

- *Spooling-Dämon* für nur exklusiv benutzbare E/A-Geräte

  → Verwaltung und Serialisierung überlappender Aufträge durch andere Prozesse

Vortragsübersicht

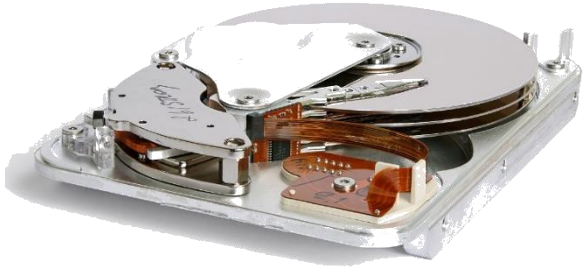
5 – Ein- und Ausgabeverwaltung

- E/A-Hardware
- E/A-Software
- Architektur für E/A-Software
- Konkrete Peripheriegeräte

Konkrete Peripheriegeräte (1)

Häufige Sekundärspeicher

magnetische Festplatte (HD):
für Dateisystem, Seitenauslagerung



optische Medien (CD, DVD, Blu-Ray):
zur kostengünstigen Verteilung von Daten



solid-state disk (SSD):
schneller als HD, ohne bewegte Teile



Festplatten-Stapel (RAID):
für hohe Zuverlässigkeit



Konkrete Peripheriegeräte (2)

Redundant Array of Inexpensive Disks (*RAID*)

logischer Datenträger aus mehreren physikalischen Datenträgern

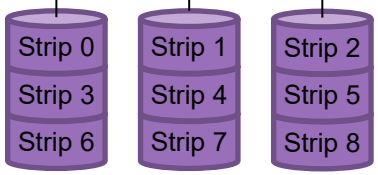
→ Daten werden auf mehrere Platten verteilt (*striping*)

→ höhere Leistung durch parallelen Zugriff auf Strips

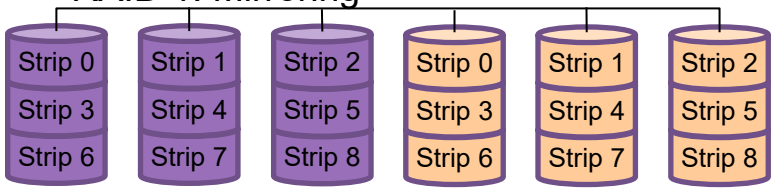
→ höhere Zuverlässigkeit durch Redundanz

→ standardisierte RAID Levels, z. B.:

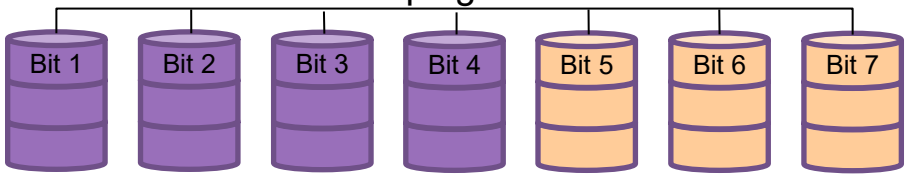
RAID 0: Striping



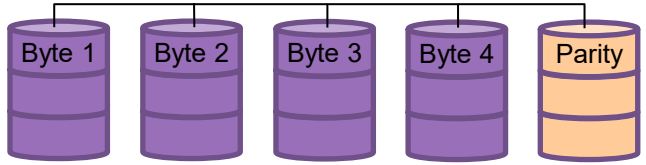
RAID 1: Mirroring



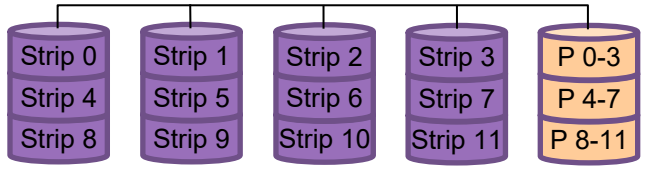
RAID 2: Bit-level striping



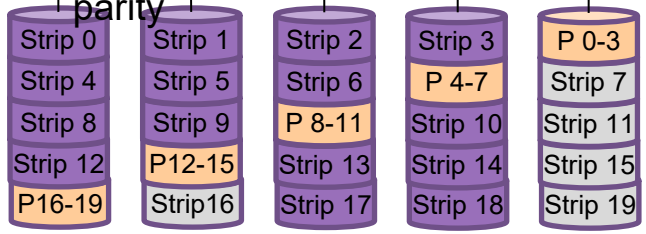
RAID 3: Byte-level striping



RAID 4: Block-level striping



RAID 5: RAID 4 + distributed parity



Zusammenfassung

Hardware

- OS steuert mit Treibern die Kontroller von Peripheriegeräten an
- Datenaustausch über Ports oder geteilten Speicher
- DMA zur Entlastung der CPU beim Kopieren von Daten
- Interrupts als Mechanismus zur Benachrichtigung des OS durch Hardware

Software

- Ziele der E/A-Software:
  Geräteunabhängigkeit, Benennungsschemata, Fehlerbehandlung,
  synchrone Programmierung, Datenbufferung, flexible Gerätenutzung
- Steuerung der E/A durch Polling, Interrupt oder DMA
- Software-Architektur für E/A ist vom Gerät zur Anwendungssoftware geschichtet,
  Schichten werden bei Eingabe und Ausgabe nacheinander durchlaufen
- Datenpufferung auf unterschiedlichen Ebenen möglich